SPRING BOOT - PRACTICAL EXAM 2

Beginner to Intermediate | Database Integration with MySQL and Spring Data JPA

Duration	Total	Pass mark	Prerequisite
90 minutes	100 marks	60%	Exam 1: REST API / HTTP methods

Student: _____ Date: _____

Scenario and Objective

Continue the Student Management REST API from Exam 1. Replace the in-memory ArrayList with a persistent MySQL database. Your application must use Spring Boot, Spring Data JPA, and HTTP methods to create, read, update, and delete student records. Data must remain available after restarting the application.

Permitted: Java 17+, Spring Boot 3.x, Maven, MySQL, Postman. You may refer to official documentation. Not required: authentication, frontend UI, Docker, or advanced architecture.

Section A - Database and Project Setup (15 marks)

Question 1: Configure the database (15 marks)

• Create a MySQL database called student_exam_db. (4 marks)

• Add the dependencies Spring Web, Spring Data JPA, and MySQL Driver. (5 marks)

• Configure the datasource URL, username, password, and JPA schema-management setting in application.properties or application.yml. (4 marks)

• Start the project and verify that the application connects to MySQL. (2 marks)

Security note: Do not commit real database passwords or secrets to version control. For the exam, local development configuration is sufficient.

Section B - Entity Mapping (15 marks)

Question 2: Convert Student into a JPA entity (15 marks)

Create or modify the Student class with these fields:

Field	Java type	Database requirement
id	Long	Primary key, generated automatically
firstName	String	Required
lastName	String	Required
email	String	Required, unique
age	Integer	Required
course	String	Required

• Use @Entity and @Id. (5 marks)

• Configure automatic primary-key generation using @GeneratedValue. (4 marks)

- Map email as unique and make the listed required fields non-nullable. (4 marks)
- Provide a no-argument constructor and suitable getters/setters. (2 marks)

Section C - Repository and Business Logic (10 marks)

Question 3: Access students through JPA (10 marks)

- Create StudentRepository extending JpaRepository<Student, Long>. (4 marks)
- Remove the ArrayList-based storage and use the repository for CRUD operations. (4 marks)
- Use dependency injection to access the repository in your controller or service. (2 marks)

Section D - REST API with Persistence (40 marks)

Question 4: Implement all six endpoints (40 marks)

HTTP method	Endpoint	Expected behavior	Marks
POST	/api/students	Save a student in MySQL; return 201 Created	8
GET	/api/students	Return all students stored in MySQL	6
GET	/api/students/{id}	Return one student; 404 if missing	6
PUT	/api/students/{id}	Replace editable student fields; 404 if missing	7
PATCH	/api/students/{id}	Update only supplied fields; 404 if missing	7
DELETE	/api/students/{id}	Delete by ID; 204 on success; 404 if missing	6

Example POST request: `POST /api/students`

```
{
  "firstName": "Mary",
  "lastName": "Smith",
  "email": "mary@example.com",
  "age": 22,
  "course": "Information Systems"
}
```

Additional rules: IDs are generated by the database and must not be supplied when creating a student. Return JSON responses. Do not create a new student when PUT or PATCH refers to an ID that does not exist. For PATCH, unchanged fields must retain their previous values.

Section E - Postman and Database Verification (10 marks)

Question 5: Demonstrate persistence (10 marks)

- Create two students using POST and show both records in MySQL. (3 marks)
- Restart the Spring Boot application; use GET to prove the records still exist. (3 marks)
- Update one student using PUT and a single field using PATCH; confirm the database values. (2 marks)
- Delete a student; confirm it is absent from MySQL and GET by ID returns 404. (2 marks)

Section F - Short Theory Questions (10 marks)

Question 6: Answer all five questions (2 marks each)

1. What is the purpose of @Entity in Spring Data JPA?

2. What does JpaRepository provide? Name two methods.

3. Explain the difference between an in-memory ArrayList and a database.

4. What is the purpose of @GeneratedValue?

5. Why should an API return 404 Not Found for an unknown student ID?

Marking Summary

Section	Marks
A - Database and project setup	15
B - Entity mapping	15
C - Repository and business logic	10
D - REST API with persistence	40
E - Testing and verification	10
F - Theory	10
TOTAL	100

Submission Checklist

• Submit the complete Maven Spring Boot source code (ZIP or Git repository link).

• Include the SQL database-creation statement and a sample configuration file without real passwords.

• Include a README with setup and run instructions.

• Submit Postman screenshots showing CRUD requests and response status codes.

• Submit a screenshot or SQL query result showing the saved student records in MySQL.

Bonus challenge (optional, not graded): Add an endpoint to find students by course, and reject duplicate email addresses with an appropriate 4xx response.